JavaScript Array Functions

Practice Questions Workbook

Welcome to the JavaScript Array Functions practice set. This workbook contains **21 hand-picked questions** covering **map**, **filter**, **reduce**, **sort**, **slice**, **splice**, **flat**, and method chaining. Try every question on your own first, then check the solutions section at the end.

How to use this workbook

1. Read each question carefully
2. Write the solution in your editor / Node.js console
3. Run it and verify the output
4. Compare with the solution at the end
5. Try to write a one-line chained version where possible

LEVEL 1 . EASY

Q1. Find the sum of all numbers.

```
const nums = [10, 20, 30, 40, 50];
```

Q2. Add 'grape' to the end and 'orange' to the beginning. Print the final array.

```
const fruits = ['apple', 'banana', 'mango'];
```

Q3. Create a new array with each number squared.

```
const nums = [1, 2, 3, 4, 5];
```

Q4. Convert all names to uppercase.

```
const names = ['naveen', 'sonu', 'priya'];
```

Q5. Find the largest number using Math.max.

```
const nums = [5, 10, 15, 20, 25];
```

LEVEL 2 . MEDIUM

Q6. Filter only the even numbers.

```
const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

Q7. Find how many students passed (passing marks >= 40).

```
const marks = [45, 78, 32, 89, 56, 90, 22];
```

Q8. Calculate the average of the array.

```
const nums = [10, 20, 30, 40, 50];
```

Q9. Reverse the array WITHOUT mutating the original.

```
const arr = [1, 2, 3, 4, 5];
```

Q10. Get only words with length greater than 3.

```
const words = ['cat', 'elephant', 'dog', 'tiger', 'rat'];
```

Q11. Remove duplicates from the array.

```
const nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5];
```

Q12. Count how many words are in the sentence.

```
const sentence = 'I love automation testing';
```

LEVEL 3 . SLIGHTLY TRICKY

Q13. Find the product of all numbers (1 x 2 x 3 x 4 x 5).

```
const nums = [1, 2, 3, 4, 5];
```

Q14. Find the sum of only odd numbers.

```
const nums = [10, 25, 30, 45, 50, 65];
```

Q15. Find the second largest number.

```
const nums = [12, 45, 78, 23, 89, 6, 34];
```

Q16. Reverse the string using array methods.

```
const str = 'naveen';
```

Q17. Flatten this array completely.

```
const arr = [1, [2, 3], [4, [5, 6]]];
```

Q18. Get only the numbers divisible by both 5 and 2.

```
const nums = [5, 10, 15, 20, 25, 30];
```

Q19. Find the sum of squares of even numbers using chaining.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

Q20. Count how many times 'apple' appears.

```
const fruits = ['apple', 'banana', 'mango', 'apple', 'banana', 'apple'];
```

BONUS CHALLENGE

Q21. In a single chain - filter numbers greater than 3, double them, and return their sum.

```
const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

Solutions

Try each question yourself before peeking. Multiple correct approaches exist - these are the cleanest one-liners.

Q1 — Sum of all numbers

```
const q1 = [10, 20, 30, 40, 50].reduce((sum, n) => sum + n, 0);
// 150
```

Q2 — Add to start and end

```
const q2 = ['apple', 'banana', 'mango'];
q2.push('grape');
q2.unshift('orange');
// ['orange', 'apple', 'banana', 'mango', 'grape']
```

Q3 — Square each number

```
const q3 = [1, 2, 3, 4, 5].map(n => n * n);
// [1, 4, 9, 16, 25]
```

Q4 — Uppercase names

```
const q4 = ['naveen', 'sonu', 'priya'].map(n => n.toUpperCase());
// ['NAVEEN', 'SONU', 'PRIYA']
```

Q5 — Largest number

```
const q5 = Math.max(...[5, 10, 15, 20, 25]);
// 25
```

Q6 — Filter even numbers

```
const q6 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    .filter(n => n % 2 === 0);
// [2, 4, 6, 8, 10]
```

Q7 — Count passing students

```
const q7 = [45, 78, 32, 89, 56, 90, 22]
    .filter(m => m >= 40).length;
// 5
```

Q8 — Average

```
const arr = [10, 20, 30, 40, 50];
const q8 = arr.reduce((sum, n) => sum + n, 0) / arr.length;
// 30
```

Q9 — Reverse without mutation

```
const q9 = [1, 2, 3, 4, 5].toReversed();
// [5, 4, 3, 2, 1]
// Original array stays unchanged
```

Q10 — Words longer than 3

```
const q10 = ['cat', 'elephant', 'dog', 'tiger', 'rat']
    .filter(w => w.length > 3);
// ['elephant', 'tiger']
```

Q11 — Remove duplicates

```
const q11 = [...new Set([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])];
// [3, 1, 4, 5, 9, 2, 6]
```

Q12 — Count words

```
const q12 = 'I love automation testing'.split(' ').length;
// 4
```

Q13 — Product of all numbers

```
const q13 = [1, 2, 3, 4, 5].reduce((acc, n) => acc * n, 1);
// 120
```

Q14 — Sum of odd numbers

```
const q14 = [10, 25, 30, 45, 50, 65]
    .filter(n => n % 2 !== 0)
    .reduce((sum, n) => sum + n, 0);
// 135
```

Q15 — Second largest

```
const q15 = [12, 45, 78, 23, 89, 6, 34]
    .toSorted((a, b) => b - a)[1];
// 78
```

Q16 — Reverse a string

```
const q16 = 'naveen'.split('').reverse().join('');
// 'neevan'
```

Q17 — Flatten completely

```
const q17 = [1, [2, 3], [4, [5, 6]]].flat(Infinity);
// [1, 2, 3, 4, 5, 6]
```

Q18 — Divisible by 5 and 2

```
const q18 = [5, 10, 15, 20, 25, 30]
    .filter(n => n % 5 === 0 && n % 2 === 0);
// [10, 20, 30]
```

Q19 — Sum of squares of evens

```
const q19 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    .filter(n => n % 2 === 0)
    .map(n => n * n)
    .reduce((sum, n) => sum + n, 0);
// 220
```

Q20 — Count occurrences

```
const q20 = ['apple', 'banana', 'mango', 'apple', 'banana', 'apple']
    .filter(f => f === 'apple').length;
// 3
```

Q21 — Bonus - chained one-liner

```
const q21 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    .filter(n => n > 3)
    .map(n => n * 2)
    .reduce((sum, n) => sum + n, 0);
// 98
```

Quick Reference Cheat Sheet

Most-used patterns for daily JavaScript / TypeScript work.

Goal	Pattern / Method
Add to end / start	`push()` / `unshift()`
Remove from end / start	`pop()` / `shift()`
Extract a portion (non-mutating)	`slice(start, end)`
Add / remove anywhere (mutating)	`splice(start, count, ...items)`
Transform each element	`map(n => ...)`
Keep matching elements	`filter(n => condition)`
Aggregate to one value	`reduce((acc, n) => ..., init)`
Check any / all match	`some()` / `every()`
Check if value exists	`includes(value)`
Find first matching element	`find(n => condition)`
Flatten nested arrays	`flat()` / `flat(Infinity)`
Sort without mutating	`toSorted((a, b) => a - b)`
Reverse without mutating	`toReversed()`
Remove duplicates	`[...new Set(arr)]`
Copy an array	`[...arr]` or `arr.slice()`
Deep copy	`structuredClone(arr)`

Keep practicing - fluency with array methods is the foundation of clean, readable test code in Playwright and beyond.

Naveen Automation Labs

QA & SDET Training | Live Batches | Free Tools
youtube.com/@naveenautomationlabs